Data Clustering Methods

1 Introduction

In daily life as well as in different fields of science, we encounter large, sometimes enormous volumes of information. A single glance is enough for humans to distinguish the shapes of objects of interest in a specific image. However, intelligent machines remain incapable of prompt and error-free object recognition due to the lack of universal algorithms applicable to every situation.

The objective of data clustering is to partition a dataset into clusters of similar data. Objects in the dataset may include bank customers, figures in a photograph, or sick and healthy individuals. Humans can effectively group one- and two-dimensional data, but three-dimensional data poses significant challenges. The problem intensifies with real-world datasets containing thousands to millions of samples. Thus, algorithms for **automatic data clustering** are essential. These algorithms produce a fixed structure of data partition, including cluster locations, shapes, and membership degrees of each sample to each cluster. Data clustering is complex due to arbitrary cluster shapes/sizes and the typically unknown number of clusters. No existing algorithm universally handles all cluster shapes.

When selecting a clustering algorithm, leverage domain knowledge. A good partition should exhibit:

- **Homogeneity in clusters**: Data within a cluster should be as similar as possible.
- **Heterogeneity between clusters**: Data across clusters should be as distinct as possible.

Similarity is often measured via distance metrics (e.g., Euclidean norm). Clusters are frequently represented by their central points. Different similarity measures yield varying cluster shapes. Since clustering lacks a "desired output signal," it is an **unsupervised learning** task. This chapter covers partitioning methods, clustering algorithms, and validity measures.

Hard and Fuzzy Partitions

Data is represented as n-dimensional vectors:

$$\mathbf{x}_k = [x_{k1}, \dots, x_{kn}]^T, \quad \mathbf{x}_k \in \mathbb{R}^n, \quad k = 1, \dots, M$$

forming the $n \times M$ matrix:

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{21} & \cdots & x_{M1} \\ x_{12} & x_{22} & \cdots & x_{M2} \\ & & \ddots & \\ x_{1n} & x_{2n} & \cdots & x_{Mn} \end{bmatrix}.$$

In medical diagnostics, rows represent symptoms and columns represent patients.

Cluster centers are vectors $\mathbf{v}_i = [v_{i1}, \dots, v_{in}]$, $i = 1, \dots, c$. The number of partitions for M objects into c clusters is:

$$\frac{1}{c!}\sum_{i=1}^{c}\binom{c}{i}(-1)^{c-i}i^{M}.$$

Example 8.1: Partitioning 100 patients into 5 clusters yields 6.57×10^{67} possible partitions.

Hard Partition

Each object belongs entirely to one cluster. Conditions:

$$\bigcup_{i=1}^{c} A_i = \mathbf{X}, \quad A_i \cap A_j = \varnothing\ (i \neq j), \quad \varnothing \subset A_i \subset \mathbf{X}.$$

The partition matrix $\mathbf{U} \in \mathbb{R}^{c\times M}$ encodes membership degrees $\mu_{ik} \in \{0,1\}$:

$$Z_1 = \left\{\mathbf{U} \in \mathbb{R}^{c\times M} \mid \mu_{ik} \in \{0,1\}, \sum_{i=1}^{c}\mu_{ik} = 1,\ 0 < \sum_{k=1}^{M}\mu_{ik} < M\right\}.$$

Example: A hard partition matrix for 3 clusters:

$$\mathbf{U} = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix}.$$

Fuzzy Partition

Objects belong to multiple clusters with degrees $\mu_{ik} \in [0,1]$ and $\sum_{i=1}^{c}\mu_{ik} = 1$:

$$Z_2 = \left\{\mathbf{U} \in \mathbb{R}^{c\times M} \mid \mu_{ik} \in [0,1], \sum_{i=1}^{c}\mu_{ik} = 1,\ 0 < \sum_{k=1}^{M}\mu_{ik} < M\right\}.$$

Example 8.3: Fuzzy partition matrix for an outlier ($\mathbf{x}_{10}$):

$$\mathbf{U} = \begin{bmatrix} 0 & 0.06 & 0.02 & 0.98 & 0.98 & 0.99 & 0.01 & 0.01 & 0 & 0.29 \\ 1 & 0.89 & 0.93 & 0.01 & 0.01 & 0.00 & 0.01 & 0.01 & 0 & 0.33 \\ 0 & 0.05 & 0.05 & 0.01 & 0.01 & 0.01 & 0.98 & 0.98 & 1 & 0.38 \end{bmatrix}.$$

Possibilistic Partition

No sum constraint on μ_{ik}, but $\exists i\ \mu_{ik} > 0$:

$$Z_3 = \left\{\mathbf{U} \in \mathbb{R}^{c\times M} \mid \mu_{ik} \in [0,1],\ \forall k\ \exists i\ \mu_{ik} > 0,\ 0 < \sum_{k=1}^{M}\mu_{ik} < M\right\}.$$

Example: Possibilistic partition for noise ($\mathbf{x}_{10}$):

$$\mathbf{U} = \begin{bmatrix} 0.01 & 0.02 & 0.01 & 0.52 & 0.39 & 0.87 & 0.01 & 0.01 & 0.01 & 0.03 \\ 0.87 & 0.44 & 0.79 & 0.04 & 0.03 & 0.03 & 0.05 & 0.04 & 0.05 & 0.12 \\ 0.01 & 0.01 & 0.02 & 0.01 & 0.01 & 0.01 & 0.53 & 0.63 & 0.79 & 0.03 \end{bmatrix}.$$

Distance Measures

- **Euclidean**:

$$D_{id} = \sqrt{\sum_{j=1}^{n} \left(x_{dj} - v_{ij}\right)^2}.$$

- **Minkowski**:

$$D_{id} = \left(\sum_{j=1}^{n} | x_{dj} - v_{ij} |^r \right)^{1/r}.$$

- **Manhattan** ($r = 1$) and **Hamming** (binary variables).
- **Weighted Euclidean**:

$$D_{id} = \sqrt{\sum_{j=1}^{n} w_j \left(x_{dj} - v_{ij}\right)^2}.$$

K-Means Clustering Algorithm: A Detailed Explanation

K-Means is an unsupervised machine learning algorithm used for partitioning a dataset into K distinct, non-overlapping clusters. It is one of the simplest and most widely used clustering techniques due to its efficiency and ease of implementation.

Rationale Behind K-Means Clustering

The goal of K-Means is to minimize the variance within each cluster while ensuring that each data point belongs to the most suitable cluster. The algorithm iteratively refines the cluster assignments based on the mean (centroid) of data points.

Key Concepts

- **Centroid**: The center of a cluster, computed as the mean of all points assigned to that cluster.

- **Cluster Assignment**: Each data point is assigned to the nearest centroid.
- **Objective Function**: K-Means minimizes the sum of squared distances between data points and their respective cluster centroids.

K-Means Algorithm Steps

The algorithm follows an iterative approach with the following steps:

Step 1: Initialize Cluster Centers

Choose K initial centroids, either randomly from the dataset or using a smarter initialization (e.g., K-Means++).

Step 2: Assign Data Points to the Nearest Centroid

For each data point x_i, assign it to the nearest centroid using the Euclidean distance formula:

$$d(x_i, c_k) = \sqrt{\sum_{j=1}^{n} (x_{ij} - c_{kj})^2}$$

where:

- x_{ij} represents the j-th feature of the i-th data point.
- c_{kj} represents the j-th feature of the k-th centroid.
- Each data point is assigned to the cluster with the closest centroid:

$$C_i = \underset{k}{\operatorname{argmin}} d(x_i, c_k)$$

Step 3: Compute New Centroids

After assigning all points, update each cluster centroid c_k by taking the mean of all data points in that cluster:

$$c_k = \frac{1}{| S_k |} \sum_{x_i \in S_k} x_i$$

where:

- S_k is the set of points assigned to cluster k.
- $| S_k |$ is the number of points in cluster k.

Step 4: Repeat Until Convergence

Repeat steps 2 and 3 until centroids no longer change significantly or a pre-defined number of iterations is reached.

Mathematical Formulation of K-Means

Objective Function

K-Means minimizes the within-cluster variance, also known as the sum of squared errors (SSE):

$$J = \sum_{k=1}^{K} \sum_{x_i \in S_k} \| x_i - c_k \|^2$$

where:

- x_i is a data point assigned to cluster k.
- c_k is the centroid of cluster k.
- S_k is the set of all points in cluster k.

The algorithm iteratively refines cluster assignments to minimize J.

Advantages and Limitations

Advantages

Simple and easy to implement.
Works well with compact, spherical clusters.

Limitations

Sensitive to initial centroid selection (can lead to local minima).
Assumes clusters are spherical and equally sized.

Bisecting K-Means Clustering Algorithm

Introduction

Bisecting K-Means is a **hierarchical variant** of the standard **K-Means algorithm** that uses a **divisive** approach to clustering. Unlike traditional K-Means, which partitions data into K clusters in one step, **Bisecting K-Means recursively splits clusters into two (bi-sectioning) until the desired number of clusters is reached**.

This algorithm **combines the advantages of both hierarchical clustering and K-Means**:

- **Better cluster quality** compared to standard K-Means.
- **Efficient computation** compared to agglomerative hierarchical clustering.
- **Robustness** in handling different cluster shapes and structures.

Rationale Behind Bisecting K-Means

Comparison with Standard K-Means

Algorithm	Approach	Strength	Weakness
K-Means	Partitions the entire dataset into K clusters in one step.	Faster for small datasets.	Can get stuck in **local optima**, sensitive to initialization.
Hierarchical Clustering	Builds a tree of clusters (bottom-up or top-down).	Produces a **hierarchical structure** useful for visualization.	Computationally expensive for large datasets.
Bisecting K-Means	Splits clusters **iteratively** into two sub-clusters.	**Better cluster quality** than K-Means and more efficient than hierarchical clustering.	Requires choosing the number of clusters manually.

Algorithm Steps

The algorithm follows a **top-down hierarchical approach**:

1. **Start with all data in one cluster**.
2. **Repeat until K clusters are formed**:
    - Choose the **largest** cluster to split.
    - Apply **K-Means with $K = 2$** to bisect the selected cluster.
    - Select the best split (based on sum of squared errors, SSE).
3. **Assign data points** to final clusters.

Mathematical Formulation

Objective Function (SSE - Sum of Squared Errors)

At each step, we minimize the total **within-cluster sum of squared errors (SSE)**:

$$SSE = \sum_{i=1}^{n} || x_i - c_j ||^2$$

where:

- n = number of data points.
- x_i = i-th data point.
- c_j = centroid of cluster j.
- $|| x_i - c_j ||^2$ = squared Euclidean distance between x_i and c_j.

K-Means Bisection Step

To split a cluster, we apply **K-Means with** $K = 2$:

1. **Initialize two centroids** c_1 and c_2 randomly.
2. **Assign points** to the closest centroid:

$$c_j = \text{argmin}_k \; || \; x_i - c_k \; ||^2$$

3. **Update centroids** iteratively:

$$c_j = \frac{1}{| \; C_j \; |} \sum_{x_i \in C_j} x_i$$

   where $| \; C_j \; |$ is the number of points in cluster C_j.

4. **Repeat** until convergence (centroids do not change significantly).

Advantages and Limitations

Advantages

Better Cluster Quality: More stable and **less sensitive to initialization** than standard K-Means.
Efficient for Large Datasets: Reduces the computational cost of hierarchical clustering.
Flexible: Can control granularity of clusters easily.

Limitations

Requires Selecting K **Manually**: Like K-Means, the number of clusters must be predefined.
Can Be Computationally Expensive: Still requires multiple K-Means runs.
Sensitive to Cluster Selection: Choosing the **largest SSE cluster** at each step may not always be optimal.

K-Means++ Clustering Algorithm: A Detailed Explanation

Introduction

K-Means++ is an improved version of the **K-Means** clustering algorithm that provides a better initialization strategy for selecting the initial cluster centroids. The primary goal of K-Means++ is to reduce the chances of poor clustering due to **bad centroid initialization** in standard K-Means.

Rationale Behind K-Means++

Problems with Standard K-Means Initialization

In standard K-Means, centroids are initialized randomly, which can cause:

- **Poor Convergence:** Randomly chosen centroids may lead to slow or failed convergence.
- **Suboptimal Clusters:** Bad initial centroids can result in highly imbalanced clusters.
- **Local Minima:** The algorithm may get stuck in a poor solution due to bad initialization.

How K-Means++ Improves Initialization

K-Means++ selects the initial centroids **in a more intelligent way** by ensuring that:

1. The first centroid is chosen randomly from the dataset.
2. Subsequent centroids are chosen **with a probability proportional to their squared distance from existing centroids**.
3. This ensures that centroids are well-separated, leading to **faster convergence** and **better clustering quality**.

K-Means++ Algorithm Steps

K-Means++ follows these steps:

Step 1: Choose First Centroid Randomly

- Pick the first centroid c_1 randomly from the dataset.

Step 2: Compute Distances for Next Centroid Selection

- For each remaining data point x_i, compute its squared distance from the closest selected centroid:

$$D(x_i)^2 = \min_{c_j \in C} d(x_i, c_j)^2$$

where:

  - $d(x_i, c_j)$ is the Euclidean distance between data point x_i and the nearest already selected centroid c_j.
  - C is the set of currently selected centroids.

Step 3: Select Next Centroid Probabilistically

- Select the next centroid c_k randomly, but with a probability proportional to $D(x_i)^2$:

$$P(x_i) = \frac{D(x_i)^2}{\sum_{x_j \in X} D\left(x_j\right)^2}$$

This means that points **farther from existing centroids** have a **higher chance of being selected**.

Step 4: Repeat Until K Centroids are Selected

- Repeat steps 2 and 3 until K centroids have been chosen.

Step 5: Run Standard K-Means

- Use the selected centroids as the starting points for the standard **K-Means algorithm**.

Mathematical Formulation

Distance Calculation

For each data point x_i, compute the squared Euclidean distance to the nearest selected centroid:

$$D(x_i)^2 = \min_{c_j \in C} \sum_{m=1}^{M} \left(x_{im} - c_{jm}\right)^2$$

where:

- x_{im} and c_{jm} represent the m-th feature of the data point and centroid, respectively.
- C is the set of currently chosen centroids.

Probability of Selecting Next Centroid

Each data point x_i is selected as the next centroid with probability:

$$P(x_i) = \frac{D(x_i)^2}{\sum_{x_j \in X} D\left(x_j\right)^2}$$

This ensures **diverse** and **well-separated** centroid selection.

Advantages and Limitations

Advantages

Better Initialization: Reduces the risk of poor cluster formation.
Faster Convergence: Fewer iterations required compared to random initialization.
More Accurate Clusters: Produces more natural and stable clusters.

Limitations

Slightly More Expensive Initialization
Does Not Guarantee Optimal Clustering: Still dependent on K choice.

K-Medoids Clustering Algorithm (PAM: Partitioning Around Medoids) - A Detailed Explanation

Introduction

K-Medoids is a clustering algorithm closely related to **K-Means**, but instead of using the mean (centroid) as the representative of a cluster, it uses **medoids**—actual data points that minimize the total dissimilarity between themselves and all other points in the cluster.

The most common implementation of K-Medoids is the **PAM (Partitioning Around Medoids) algorithm**, developed by Kaufman and Rousseeuw (1987).

Rationale Behind K-Medoids Clustering

K-Means is highly sensitive to **outliers** since it uses the mean as the cluster center. K-Medoids, on the other hand, uses **medoids**, which are actual **data points**, making it more **robust to outliers** and **more interpretable** in real-world applications.

Key Concepts

- **Medoid**: A representative point in a cluster that minimizes the sum of dissimilarities within the cluster.
- **Cluster Assignment**: Each data point is assigned to the medoid with the smallest dissimilarity.
- **Objective Function**: K-Medoids minimizes the sum of absolute differences (or any chosen distance metric) rather than squared differences like K-Means.

K-Medoids Algorithm Steps (PAM)

Step 1: Initialize Medoids

- Select K initial medoids randomly from the dataset.

Step 2: Assign Each Data Point to the Nearest Medoid

- Assign each data point to the closest medoid based on a chosen distance metric (e.g., **Manhattan** or **Euclidean distance**).

Step 3: Swap and Optimize Medoids

- For each non-medoid data point, compute the **cost of swapping** it with the medoid.
- If a swap **reduces the total clustering cost**, perform the swap.
- Repeat until no further swaps improve the clustering.

Step 4: Repeat Until Convergence

- Stop when the medoids remain unchanged or a pre-defined number of iterations is reached.

Mathematical Formulation

Distance Metric

K-Medoids can use any distance metric, but a common choice is the **Manhattan Distance**:

$$d(x_i, x_j) = \sum_{m=1}^{M} | x_{im} - x_{jm} |$$

where:

- x_i and x_j are two data points with M features each.
- $| x_{im} - x_{jm} |$ represents the absolute difference in the m-th feature.

Alternatively, **Euclidean Distance** can be used:

$$d(x_i, x_j) = \sqrt{\sum_{m=1}^{M} (x_{im} - x_{jm})^2}$$

Objective Function

K-Medoids minimizes the **sum of distances** between each point and its assigned medoid:

$$J = \sum_{k=1}^{K} \sum_{x_i \in S_k} d\,(x_i, m_k)$$

where:

- K is the number of clusters.
- S_k is the set of points assigned to cluster k.

- m_k is the medoid of cluster k.
- $d(x_i, m_k)$ is the distance between point x_i and medoid m_k.

Cost of Swapping a Medoid

For each non-medoid data point x_j, compute the change in total distance if it replaces an existing medoid m_k:

$$\Delta J = \sum_{x_i \in S_k} \left[d(x_i, x_j) - d(x_i, m_k) \right]$$

If $\Delta J < 0$, then swapping reduces the total clustering cost, and the swap is accepted.

Advantages and Limitations

Advantages

Robust to outliers since medoids are actual data points.
More interpretable since cluster centers are real data points.

Limitations

Computationally expensive compared to K-Means.
Sensitive to the choice of K (requires domain knowledge or Elbow Method).

C-Means Clustering Algorithm (Fuzzy C-Means - FCM)

Introduction

C-Means Clustering, commonly referred to as **Fuzzy C-Means (FCM)**, is a soft clustering algorithm that allows data points to belong to multiple clusters with varying degrees of membership. Unlike **K-Means**, which forces each data point into precisely one cluster, **FCM assigns each point a membership value** between **0 and 1**, representing its degree of belonging to different clusters.

FCM is widely used in applications with **fuzzy boundaries**, such as **image segmentation, medical diagnostics, and pattern recognition**.

Rationale Behind FCM

Limitations of K-Means

- **Hard Clustering**: Each point belongs to exactly **one** cluster, which is unrealistic in many real-world scenarios.
- **Lack of Uncertainty Representation**: No indication of **how strongly a point belongs** to its assigned cluster.

Advantages of FCM

Soft Clustering: Data points can belong to multiple clusters with different degrees of membership.
Better Representation of Uncertainty: Fuzzy membership functions help **capture ambiguity** in data.
Improved Cluster Overlap Handling: Useful in cases where clusters have **fuzzy boundaries**.

The Fuzzy C-Means (FCM) Algorithm

Using **fuzzy membership values**, FCM minimises an objective function that measures **the weighted sum of squared distances** between data points and cluster centers.

Algorithm Steps

1. Initialize **K** cluster centroids randomly.
2. Assign **fuzzy membership values** to each data point.
3. Compute new centroids based on the fuzzy memberships.
4. Update membership values iteratively.
5. Repeat until convergence (change in centroids is small).

Mathematical Formulation of FCM

Objective Function

FCM minimizes the following **fuzzy objective function**:

$$J_m(U, C) = \sum_{i=1}^{n} \sum_{j=1}^{K} u_{ij}^{m} \cdot d\left(x_i, c_j\right)^2$$

where:

- n = number of data points.

- K = number of clusters.
- x_i = i-th data point.
- c_j = j-th cluster centroid.
- u_{ij} = membership of x_i in cluster j (**fuzzy membership**).
- $d(x_i, c_j)$ = Euclidean distance between x_i and c_j:

$$d(x_i, c_j) = \sqrt{\sum_{m=1}^{M} (x_{im} - c_{jm})^2}$$

- m = fuzziness parameter ($m > 1$), which controls the degree of fuzziness. A larger m makes clusters more **fuzzy**, while $m \to 1$ approaches **hard clustering**.

Membership Update Rule

Membership values are updated using:

$$u_{ij} = \frac{1}{\sum_{k=1}^{K} \left(\frac{d(x_i, c_j)}{d(x_i, c_k)} \right)^{\frac{2}{m-1}}}$$

where:

- u_{ij} represents the membership degree of x_i in cluster j.
- The denominator ensures that the sum of all membership values for a data point equals **1**:

$$\sum_{j=1}^{K} u_{ij} = 1, \quad \forall i$$

Centroid Update Rule

Cluster centroids are updated based on the **weighted mean** of all data points:

$$c_j = \frac{\sum_{i=1}^{n} u_{ij}^m \cdot x_i}{\sum_{i=1}^{n} u_{ij}^m}$$

This ensures that centroids are influenced more by points with **higher membership values**.

Advantages and Limitations

Advantages

Soft Clustering: Points can belong to multiple clusters.
Handles Overlapping Clusters: Suitable for real-world datasets with uncertain boundaries.
Better than K-Means in Some Cases: Works well when clusters are not clearly separated.

Limitations

Computationally Expensive: More iterations due to **fuzzy membership updates**.
Sensitive to Fuzziness Parameter (m): A poor choice of m can lead to **bad clustering results**.
Does Not Guarantee Optimal Clusters: Can still get stuck in **local minima**.

Possibilistic C-Means (PCM) Clustering Algorithm

Introduction

Possibilistic C-Means (PCM) is a clustering algorithm designed to address the limitations of **Fuzzy C-Means (FCM)**. Unlike **FCM**, which assigns each data point a membership value that sums to 1 across all clusters, **PCM allows data points to belong to clusters independently**, meaning a data point **can have a high membership in multiple clusters or none at all**.

PCM is especially useful in situations where **outliers exist** or when the data contains **overlapping clusters**, as it does not force every data point into one of the predefined clusters.

Rationale Behind PCM

Limitations of Fuzzy C-Means (FCM)

- **Constraint Issue**: FCM enforces $\sum_{j=1}^{K} u_{ij} = 1$ for all data points, meaning **every point must belong to some cluster**.
- **Outlier Sensitivity**: FCM tends to assign small but nonzero membership values to **outliers**, making the model less robust.
- **Clusters Can Merge**: When clusters are highly overlapping, FCM tends to merge them rather than maintain distinct groups.

Advantages of PCM

Independent Membership Values: Each point has a **possibility degree** for each cluster instead of a forced sum constraint.

Handles Outliers: PCM can assign **very low membership values** to outliers, making it more robust.
Better Separation of Clusters: Works well in datasets with **overlapping clusters**.

Mathematical Formulation of PCM

Objective Function

PCM minimizes the following **possibilistic objective function**:

$$J_m(U,C) = \sum_{j=1}^{K}\sum_{i=1}^{n} u_{ij}^{m}\, d\left(x_i, c_j\right)^2 + \sum_{j=1}^{K}\gamma_j \sum_{i=1}^{n}\left(1-u_{ij}\right)^m$$

where:

- n = number of data points.
- K = number of clusters.
- x_i = i-th data point.
- c_j = centroid of cluster j.
- u_{ij} = **possibilistic membership value** of x_i in cluster j.
- $d\left(x_i, c_j\right)$ = Euclidean distance between x_i and c_j:

$$d\left(x_i, c_j\right) = \sqrt{\sum_{m=1}^{M}\left(x_{im} - c_{jm}\right)^2}$$

- m = fuzziness parameter ($m > 1$), controls how fuzzy the clustering is.
- γ_j = cluster-specific constant that controls **membership strength**.

Membership and Centroid Update Rules

Updating Membership Values

The PCM membership function is different from FCM and is given by:

$$u_{ij} = \frac{1}{1 + \left(\frac{d(x_i, c_j)^2}{\gamma_j} \right)^{\frac{1}{m-1}}}$$

where:

- u_{ij} is independent for each cluster (does not sum to 1).
- γ_j is computed as:

$$\gamma_j = \eta \cdot \frac{\sum_{i=1}^{n} u_{ij}^m \, d(x_i, c_j)^2}{\sum_{i=1}^{n} u_{ij}^m}$$

  where η is a user-defined parameter (typically between **0.1 and 1**).

Updating Cluster Centroids

Centroids are updated using the **weighted mean**:

$$c_j = \frac{\sum_{i=1}^{n} u_{ij}^m \, x_i}{\sum_{i=1}^{n} u_{ij}^m}$$

This ensures that centroids are influenced more by points with **higher membership values**.

Advantages and Limitations

Advantages

Handles Outliers Well: Unlike FCM, PCM does not force every point into a cluster.
Independent Membership: Each point has a **possibility degree** for each cluster rather than a forced sum constraint.
Better Cluster Separation: Useful in applications like **medical imaging, anomaly detection, and document clustering**.

Limitations

Sensitive to Parameter η: Poor choices of η can lead to **incorrect clustering**.
Computationally Expensive: More iterations are needed compared to FCM.
Clusters Can Overlap: If γ_j values are not well chosen, PCM may fail to **clearly separate clusters**.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) Algorithm

Introduction

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a **density-based clustering algorithm** that identifies clusters based on the **density of data points** in a given space. Unlike K-Means and hierarchical clustering, **DBSCAN does not require the number of clusters to be predefined** and can discover clusters of **arbitrary shape** while distinguishing **noise (outliers)**.

It is particularly useful for:

Detecting clusters of varying shapes and densities
Handling noise (outliers) effectively
Clustering large datasets efficiently

Rationale Behind DBSCAN

Limitations of Other Clustering Methods

- **K-Means**: Assumes clusters are spherical and requires the number of clusters K as input.
- **Hierarchical Clustering**: Computationally expensive and requires choosing a linkage method.
- **Fuzzy C-Means (FCM)**: Struggles with noise and overlapping clusters.

Advantages of DBSCAN

Does not require specifying K (the number of clusters).
Can detect clusters of arbitrary shape (e.g., elongated, curved).
Handles noise/outliers naturally by labeling them as **noise points**.
Works well on large datasets with spatial data.

Mathematical Formulation

DBSCAN relies on two key parameters:

- ε (**epsilon**): The **radius** within which points are considered neighbors.
- **MinPts**: The **minimum number of points** required to form a dense region (a cluster).

Definitions

1. **Core Point**: A point p is a core point if at least **MinPts** points (including p itself) are within its ε**-neighborhood**:

$$N_\varepsilon(p) = \{q \in X \mid d(p,q) \leq \varepsilon\}$$

   where $d(p,q)$ is the Euclidean distance.

2. **Border Point**: A point that is **not a core point** but lies **within the neighborhood of a core point**.

3. **Noise Point (Outlier):** A point that is **neither a core point nor a border point**.

DBSCAN Algorithm Steps

1. **For each point p in the dataset**:
   - If p is already classified, skip it.
   - Find all points **within distance** ε of p (neighbors).
   - If p is a **core point** (i.e., has at least **MinPts** neighbors):
     - Create a new cluster and recursively add **reachable** points.
   - If p is a **border point**, assign it to the nearest cluster.
   - If p is a **noise point**, label it as an outlier.

Advantages and Limitations

Advantages

Identifies clusters of arbitrary shape.
Automatically detects noise/outliers.
Does not require specifying K like K-Means.
Efficient for large spatial datasets.

Limitations

Choosing ε **and MinPts**: Bad choices can lead to poor clustering.
Not suitable for high-dimensional data due to **curse of dimensionality**.
Sensitive to density variation: If clusters have varying densities, DBSCAN may fail.